BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

INTRODUCTION TO WEB PROGRAMMING

Unit 8

Advanced JavaScript

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3-4
	1.1 Learning Objectives	-	-	
2	Literals	-	-	5-7
3	Functions	-	-	8-10
4	Objects	-	-	10-12
5	Arrays	-	-	13-14
6	Built in Objects	-	-	15-147
7	JavaScript Debuggers	-	-	18-23
8	Summary	-	-	23
9	Questions	-	1	24-25
10	Answers	-	-	25-26

1. INTRODUCTION

In the dynamic and ever-evolving world of web development, JavaScript stands as a cornerstone programming language. It empowers developers to create interactive and responsive web applications that captivate users worldwide. This chapter delves into the fundamental building blocks of JavaScript, offering you a comprehensive understanding of key concepts that form the bedrock of the language.We begin our journey with JavaScript literals, the fundamental building blocks for representing data. Whether it's a simple number, a string of text, or a complex data structure, literals are at the heart of every JavaScript program. We'll explore different types of literals, such as numeric, string, boolean, and more, and discover how to use them effectively in your code.Functions are the workhorses of JavaScript, allowing you to encapsulate logic and execute it repeatedly. In this chapter, we'll delve deep into functions, exploring their syntax, parameters, return values, and how to create reusable code blocks that make your programs more efficient and maintainable.JavaScript is known for its versatility when it comes to handling data. Objects play a pivotal role in this regard. We'll investigate how objects are defined and used in JavaScript, enabling you to model real-world entities and organize data effectively. You'll gain insights into object properties, methods, and prototypes.

Arrays provide a structured way to store and manipulate collections of data in JavaScript. We'll learn how to create arrays, access elements, modify data, and employ various array methods to streamline common operations, making your code more efficient and readable.JavaScript comes bundled with a rich set of built-in objects and functions that extend its capabilities. This chapter explores some of the essential built-in objects, such as Date, Math, and JSON, demonstrating how they can simplify complex tasks and enhance your code's functionality.

Debugging is an indispensable skill for any developer. We'll introduce you to JavaScript debuggers, essential tools that help you identify and resolve errors in your code. Learn how to set breakpoints, inspect variables, and step through code execution to track down and fix issues efficiently. By the end of this chapter, you'll have a solid foundation in JavaScript's core concepts, equipping you with the knowledge and skills needed to create robust web applications and tackle more advanced topics in the world of web development. So, let's

embark on this journey into the heart of JavaScript, where you'll discover the power and versatility of this dynamic language.

1.1 Learning objectives

By the end of this chapter you will be able to:

- *Differentiate between primitive and composite data types and provide examples of each.*
- *Interpret and analyze JavaScript function declarations and expressions, including parameters and return values.*
- *Implement common object manipulation techniques, such as adding and removing*
- *Evaluate scenarios where built-in objects can simplify code development and enhance efficiency.*
- *Design a debugging strategy to identify and resolve common programming errors in JavaScript code.*

2. LITERALS

Literals in JavaScript are fundamental representations of values within your code. They are used to define data directly in your scripts without the need for complex expressions or function calls. JavaScript supports various types of literals, including numeric literals, string literals, boolean literals, object literals, array literals, and more. Let's dive into each of these types with their syntax and examples:

1. *Numeric Literals:*

Numeric literals represent numbers in JavaScript. They can be integers or floating-point numbers.

Syntax:

```
let integerLiteral = 42;        // Integer literal
let floatLiteral = 3.14;        // Floating-point literal
```

Example:

```
let age = 30;
let price = 19.99;
```

2. *String Literals:*

String literals represent text or sequences of characters enclosed in single ('') or double ("") quotes.

Syntax:

```
let singleQuoted = 'Hello, World!';    // Single-quoted string literal
let doubleQuoted = "JavaScript";       // Double-quoted string literal
```

Example:

```
let greeting = 'Hello, there!';
let message = "This is a string.";
```

3. *Boolean Literals:*

Boolean literals represent two values: `true` or `false`.

Syntax:

```
let isTrue = true;
let isFalse = false;
```

Example:

```
let isUserLoggedIn = true;
let hasPermission = false;
```

4. *Object Literals:*

Object literals are used to define objects with properties and values.

Syntax:

```
let person = {
  firstName: 'John',
  lastName: 'Doe',
  age: 30
};
```

Example:

```
let car = {
  make: 'Toyota',
  model: 'Camry',
  year: 2022
};
```

5. *Array Literals:*

Array literals are used to define arrays, which are ordered lists of values enclosed in square brackets [].

Syntax:

```
let colors = ['red', 'green', 'blue'];
```

Example:

```
let numbers = [1, 2, 3, 4, 5];
```

6. Regular Expression Literals:

Regular expression literals are used to define regular expressions, which are patterns used for string matching and manipulation.

Syntax:

```
let regex = /pattern/;
```

Example:

```
let emailPattern = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}$/;
```

7. Null Literal:

The `null` literal represents the intentional absence of any object value.

Syntax:

```
let emptyValue = null;
```

Example:

```
let noValue = null;
```

8. Undefined Literal:

The `undefined` literal represents a variable that has been declared but hasn't been assigned a value.

Syntax:

```
let uninitializedVariable;
```

Example:

```
let x;
console.log(x); // Outputs: undefined
```

These are the most common types of literals in JavaScript. They allow you to define data directly in your code, making it more readable and efficient. Understanding how to use literals is essential for building JavaScript programs.

3. FUNCTIONS

Functions are one of the fundamental building blocks of JavaScript. They are reusable blocks of code that can take input, perform a specific task, and return a result. Functions allow you to encapsulate logic, make your code more organized, and promote code reusability. Let's explore JavaScript functions in detail, including their syntax and examples:

Function Syntax:

```
function functionName(parameters) {
  // Function body
  // Code to be executed when the function is called
  return result; // Optional: Return a value
}
```

- `function`: The keyword used to declare a function.
- `functionName`: The name of the function, which you can choose. It should follow naming conventions and be descriptive of its purpose.
- `parameters`: Optional parameters (inputs) that the function can accept. You can have zero or more parameters enclosed in parentheses.
- `{}`: Curly braces denote the function body, where you write the code to be executed when the function is called.
- `return`: An optional keyword used to specify the value that the function should return. If omitted, the function returns `undefined`.

Function Example without Parameters:

```
function greet() {
  console.log('Hello, world!');
}
greet(); // Call the function
// Output: Hello, world!
```

Function Example with Parameters:

```
function add(x, y) {
  return x + y;
```

```
}
let sum = add(5, 3); // Call the function and store the result in 'sum'
console.log(sum); // Output: 8
```

Function Expression Syntax:

You can also define functions using function expressions, which allow you to assign functions to variables. Function expressions are often used for anonymous functions (functions without names).

```
const functionName = function(parameters) {
  // Function body
  // Code to be executed when the function is called
  return result; // Optional: Return a value
};
```

Function Expression Example:

```
const multiply = function(x, y) {
  return x * y;
};
let product = multiply(4, 6);
console.log(product); // Output: 24
```

Arrow Function Syntax (ES6):

Arrow functions provide a concise way to define functions in ES6. They are often used for simple, one-liner functions.

```
const functionName = (parameters) => expression;
```

Arrow Function Example:

```
const square = (x) => x * x;
let result = square(7);
console.log(result); // Output: 49
```

Function Declarations vs. Function Expressions vs. Arrow Functions:

- Function Declarations: Are hoisted, meaning they can be called before they are defined in the code. They have a function name.

- Function Expressions: Are not hoisted. They are defined when the program reaches that line of code. They can be anonymous or have a name.
- Arrow Functions: Are concise and have lexical scoping for `this`. They are suitable for simple functions.

Functions are an essential part of JavaScript, enabling you to structure and modularize your code effectively, making it easier to read, maintain, and reuse.

4. OBJECTS

In JavaScript, objects are a fundamental data structure that allows you to represent and organize data as key-value pairs. Objects are used extensively in JavaScript to model real-world entities, group related data, and create complex data structures. Understanding objects is crucial for effective JavaScript programming. Let's delve into objects in detail, including their syntax and examples:

Object Syntax:

Objects are defined using curly braces `{}` and consist of zero or more key-value pairs separated by colons (`:`). The keys (also called properties) are strings (or symbols in ES6+), and the values can be of any data type, including other objects.

```
let myObject = {
  key1: value1,
  key2: value2,
  // ...more key-value pairs...
};
```

- `myObject`: This is the variable that holds the object.
- `key1`, `key2`: These are the property names or keys. They can be strings or symbols.
- `value1`, `value2`: These are the corresponding values associated with the keys. Values can be of any data type, including strings, numbers, arrays, functions, or even other objects.

Object Example:

Here's a simple JavaScript object representing information about a person:

```
let person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  isStudent: false,
};
```

In this example:

- `person` is an object variable.
- `firstName`, `lastName`, `age`, and `isStudent` are keys or properties.
- `"John"`, `"Doe"`, `30`, and `false` are the corresponding values associated with those properties.

You can access the values in an object using dot notation (`objectName.propertyName`) or bracket notation (`objectName['propertyName']`).

```
console.log(person.firstName); // Output: John
console.log(person['lastName']); // Output: Doe
```

Adding and Modifying Properties:

You can add or modify properties in an object by simply assigning new values to them:

```
person.country = "USA"; // Adding a new property
person.age = 31; // Modifying an existing property
```

Nested Objects:

Objects can contain other objects as their properties, creating a hierarchy of data. This is often used for more complex data structures.

```
let student = {
  firstName: "Alice",
  lastName: "Johnson",
  contact: {
    email: "alice@example.com",
    phone: "123-456-7890",
```

```
  },
};
```

In this example, `student` contains a nested object `contact`.

Methods:

Objects can also have methods, which are functions stored as properties. Methods can perform actions associated with the object.

```
let circle = {
  radius: 5,
  calculateArea: function () {
    return Math.PI  this.radius  this.radius;
  },
};
console.log(circle.calculateArea()); // Output: 78.54
```

ES6 Object Shorthand:

In modern JavaScript, ES6 introduced shorthand property names and methods, making object creation more concise:

```
let name = "John";
let age = 30;
let person = {
  name,
  age,
  greet() {
    console.log(`Hello, my name is ${this.name} and I'm ${this.age} years old.`);
  },
};

person.greet(); // Output: Hello, my name is John and I'm 30 years old.
```

JavaScript objects are versatile and powerful, allowing you to structure and manage data in a flexible and organized way. They are essential for many aspects of JavaScript development, including managing state in web applications and working with data from APIs.

5. ARRAYS

Arrays in JavaScript are ordered, indexed collections of values. They are versatile and commonly used to store and manipulate lists of data. JavaScript arrays can contain elements of different data types, making them flexible for various use cases. Let's explore arrays in detail, including their syntax and examples:

Array Syntax:

Arrays are defined using square brackets `[]`, and they can contain zero or more elements separated by commas.

let myArray = [element1, element2, element3, / ...more elements... /];

- `myArray`: This is the variable that holds the array.
- `element1`, `element2`, `element3`: These are the individual elements of the array, which can be of any data type.

Array Example:

Here's a simple JavaScript array containing a list of fruits:

let fruits = ["apple", "banana", "cherry", "date"];

In this example:

- `fruits` is an array variable.
- `"apple"`, `"banana"`, `"cherry"`, and `"date"` are the elements of the array.

You can access elements in an array using square brackets with the index (starting from 0):

console.log(fruits[0]); // Output: apple

console.log(fruits[2]); // Output: cherry

Array Properties and Methods:

Arrays in JavaScript come with various built-in properties and methods that allow you to manipulate and work with the data stored in them. Here are some common array properties and methods:

- `length`: Returns the number of elements in the array.

console.log(fruits.length); // Output: 4

- `push()`: Adds elements to the end of the array.

```
fruits.push("grape"); // Adds "grape" to the end
console.log(fruits); // Output: ["apple", "banana", "cherry", "date", "grape"]
```

- `pop()`: Removes the last element from the array and returns it.

```
let lastFruit = fruits.pop(); // Removes "grape" and stores it in 'lastFruit'
console.log(fruits); // Output: ["apple", "banana", "cherry", "date"]
console.log(lastFruit); // Output: grape
```

- `shift()`: Removes the first element from the array and returns it.

```
let firstFruit = fruits.shift(); // Removes "apple" and stores it in 'firstFruit'
console.log(fruits); // Output: ["banana", "cherry", "date"]
console.log(firstFruit); // Output: apple
```

- `unshift()`: Adds elements to the beginning of the array.

```
fruits.unshift("kiwi", "grapefruit"); // Adds "kiwi" and "grapefruit" to the beginning
console.log(fruits); // Output: ["kiwi", "grapefruit", "banana", "cherry", "date"]
```

- `splice()`: Modifies an array by adding, removing, or replacing elements.

```
fruits.splice(2, 1, "strawberry"); // Removes 1 element from index 2 and adds "strawberry"
console.log(fruits); // Output: ["kiwi", "grapefruit", "strawberry", "cherry", "date"]
```

- `concat()`: Combines two or more arrays to create a new array.

```
let moreFruits = ["pear", "orange"];
let combined = fruits.concat(moreFruits);
console.log(combined); // Output: ["kiwi", "grapefruit", "strawberry", "cherry", "date", "pear", "orange"]
```

These are just a few of the many methods and properties available for working with arrays in JavaScript. Arrays are incredibly versatile and form the basis for many data manipulation tasks in JavaScript programming.

6. BUILT IN OBJECTS

Built-in objects in JavaScript are a set of objects that are part of the core JavaScript language and provide essential functionality for various tasks. These objects are available globally in any JavaScript environment, whether it's a web browser or a server-side JavaScript runtime like Node.js. Here, we'll discuss some of the most commonly used built-in objects, their syntax, and provide examples.

1. *Math Object*

The `Math` object provides mathematical operations and constants.

Syntax:

Math.propertyOrMethod

Example:

```
console.log(Math.PI);     // Output: 3.141592653589793
console.log(Math.sqrt(16)); // Output: 4
console.log(Math.random()); // Output: A random number between 0 and 1
```

2. *Date Object*

The `Date` object allows you to work with dates and times.

Syntax:

new Date();

Example:

```
const currentDate = new Date();
console.log(currentDate); // Output: Current date and time
```

3. *String Object*

JavaScript treats strings as primitive values, but you can use methods of the `String` object to manipulate strings.

Syntax:

```
let myString = 'Hello, World!';
myString.propertyOrMethod;
```

Example:

```
let greeting = 'Hello, World!';
console.log(greeting.length);          // Output: 13
console.log(greeting.toUpperCase());     // Output: HELLO, WORLD!
console.log(greeting.indexOf('World'));   // Output: 7
```

4. *Array Object*

Arrays in JavaScript are objects that provide methods for working with lists of data.

Syntax:

```
let myArray = [element1, element2, / ... /];
myArray.propertyOrMethod;
```

Example:

```
let numbers = [1, 2, 3, 4, 5];
console.log(numbers.length);       // Output: 5
console.log(numbers.reverse());     // Output: [5, 4, 3, 2, 1]
console.log(numbers.join('-'));    // Output: "5-4-3-2-1"
```

5. *JSON Object*

The `JSON` (JavaScript Object Notation) object provides methods for parsing JSON strings and converting JavaScript objects to JSON.

Syntax:

```
JSON.parse(jsonString);
JSON.stringify(jsonObject);
```

Example:

```
let jsonData = '{"name": "John", "age": 30}';
let parsedData = JSON.parse(jsonData);
console.log(parsedData.name); // Output: "John"
let person = { name: 'Alice', age: 25 };
let jsonString = JSON.stringify(person);
console.log(jsonString); // Output: '{"name":"Alice","age":25}'
```

6. *RegExp Object*

The `RegExp` (Regular Expression) object allows you to work with regular expressions for pattern matching in strings.

Syntax:

```
new RegExp(pattern, flags);
```

Example:

```
let pattern = /[0-9]+/;
let str = 'There are 42 apples in the basket.';
console.log(pattern.test(str)); // Output: true
```

These built-in objects provide a wide range of functionality that is crucial for many JavaScript applications. Understanding and effectively using these objects can greatly enhance your ability to work with data and perform various tasks in JavaScript.

7. JAVASCRIPT DEBUGGERS

JavaScript debuggers are tools that help developers identify and resolve issues, bugs, and errors in their JavaScript code. Debuggers provide an interactive environment for stepping through code execution, inspecting variables, and analyzing program behavior. In this explanation, we'll cover JavaScript debuggers in detail, including syntax and examples.

Using Debuggers in Browsers (Browser DevTools)

Most modern web browsers come with built-in developer tools that include a debugger. You can access these tools by pressing `F12` or `Ctrl+Shift+I` (Windows/Linux) or `Cmd+Option+I` (macOS) to open the browser's Developer Console.

Here's a breakdown of common debugging actions using browser DevTools:

1. *Setting Breakpoints:*
   - Place breakpoints by clicking on the line number in the source code or using the `debugger;` statement in your JavaScript code.

```
function calculateSum(a, b) {
  debugger; // Breakpoint
  return a + b;
}
```

2. *Stepping Through Code:*
   - Use buttons like "Step Into," "Step Over," and "Step Out" to control code execution line by line.
3. *Inspecting Variables:*
   - View the values of variables and objects in the current scope.
4. *Watching Expressions:*
   - Add expressions to watch their values change during debugging.
5. *Call Stack and Scope:*
   - Observe the call stack to track function calls and inspect variable scopes.
6. *Console Output:*
   - Output debugging information using `console.log()` statements.

Here's an example of debugging a function in the browser's DevTools:

```
function calculateSum(a, b) {
  let result = a + b;
  console.log("Sum is:", result);
  return result;
}
let x = 5;
let y = 7;
let sum = calculateSum(x, y);
```

Using Debuggers in Node.js

You can also debug JavaScript code in Node.js using the built-in Node.js Debugger (Node Inspector).

1. *Enable Debugger:*
   - Start your Node.js script with the `--inspect` flag to enable debugging. For example:

```
node --inspect myscript.js
```

2. *Debugging Actions:*
   - Use breakpoints, inspect variables, and control code execution in a similar manner as browser DevTools.

Here's a simple example of debugging a Node.js script:

```
function calculateSum(a, b) {
  let result = a + b;
  console.log("Sum is:", result);
  return result;
}
let x = 5;
let y = 7;
let sum = calculateSum(x, y);
```

Using Debuggers in Visual Studio Code (VSCode)

Visual Studio Code (VSCode) provides a powerful debugging environment for JavaScript. You can set breakpoints, step through code, inspect variables, and use various debugging features.

1. *Launch Configuration:*
   - Create a `.vscode/launch.json` configuration file for your project.
2. *Setting Breakpoints:*
   - Click on the left margin next to the line number to set breakpoints.
3. *Debugging Actions:*
   - Use VSCode's debugging controls, including "Run" and "Debug," to start debugging sessions.

Here's a simplified example of debugging in VSCode:

```
function calculateSum(a, b) {
  let result = a + b;
  console.log("Sum is:", result);
  return result;
}
let x = 5;
let y = 7;
let sum = calculateSum(x, y);
```

Debugging is a crucial skill for developers, as it helps identify and fix issues in your code effectively. Learning to use JavaScript debuggers in your preferred environment (browser, Node.js, or IDE) can significantly improve your development workflow.

Here's an example program that incorporates various JavaScript concepts such as literals, functions, objects, arrays, built-in objects, and JavaScript debuggers:

```
// Define an object to represent a person
let person = {
  firstName: 'John',
  lastName: 'Doe',
  age: 30,
```

```
  hobbies: ['reading', 'golf', 'cooking'],
  address: {
    street: '123 Main St',
    city: 'Anytown',
    zipCode: '12345'
  },
  sayHello: function() {
    console.log(`Hello, my name is ${this.firstName} ${this.lastName}.`);
  }
};
// Create an array of book titles
let books = ['The Great Gatsby', 'To Kill a Mockingbird', '1984', 'The Hobbit'];
// Function to calculate the sum of an array of numbers
function calculateSum(numbers) {
  let sum = 0;
  for (let number of numbers) {
    sum += number;
  }
  return sum;
}
// Calculate the sum of numbers in an array
let numbers = [5, 10, 15, 20];
let total = calculateSum(numbers);
// Display the results
console.log('Person Object:');
console.log(person);
console.log('Books Array:');
console.log(books);
console.log('Total Sum:');
console.log(total);
// Using JavaScript Debugger
debugger; // Set a breakpoint here
```

// Invoke the sayHello method of the person object

person.sayHello();

The output of the program would look like this :

```
node /tmp/og4STfQUtd.js
Person Object:
{
  firstName: 'John',
  lastName: 'Doe',
  age: 30,
  hobbies: [ 'reading', 'golf', 'cooking' ],
  address: { street: '123 Main St', city: 'Anytown', zipCode: '12345' },
  sayHello: [Function: sayHello]
}
Books Array:
[ 'The Great Gatsby', 'To Kill a Mockingbird', '1984', 'The Hobbit' ]
Total Sum:
50
Hello, my name is John Doe.
Welcome to Programiz!
```

Explanation of the Program:

1. We start by defining an object called `person`, which represents a person with various properties such as `firstName`, `lastName`, `age`, `hobbies`, and `address`. It also includes a `sayHello` method that logs a greeting to the console.
2. Next, we create an array called `books` containing a list of book titles.
3. We define a function named `calculateSum` that takes an array of numbers as a parameter and calculates their sum using a `for...of` loop.
4. We create an array of numbers and calculate the sum using the `calculateSum` function.
5. We then display the `person` object, the `books` array, and the calculated `total` sum using `console.log()`.
6. To demonstrate the use of a JavaScript debugger, we insert a `debugger` statement, setting a breakpoint in the code.
7. Finally, we call the `sayHello` method of the `person` object to display a greeting.

You can run this program in a JavaScript environment such as a web browser's console or a Node.js environment with debugging support. It showcases various JavaScript concepts and allows you to use a debugger to step through the code and inspect variables, making it a practical example to learn from.

8. SUMMARY

The chapter on Literals, Functions, Objects, Arrays, Built-in Objects, and JavaScript Debuggers provides a comprehensive exploration of foundational JavaScript concepts and tools. It begins with an examination of literals, the elemental building blocks for data representation, followed by an in-depth study of functions, empowering developers to encapsulate and reuse code effectively. Objects and arrays are introduced as essential data structures, showcasing how they organize and manipulate data, while built-in objects such as Math, Date, and JSON extend the language's capabilities. The chapter culminates in a practical understanding of JavaScript debuggers, enabling developers to pinpoint and resolve issues through step-by-step code inspection, ensuring a solid foundation for effective web development.

9. QUESTIONS

Self-Assessment Questions

SELF-ASSESSMENT QUESTIONS – 1

1. What is a string literal in JavaScript, and how is it defined?
2. Explain the purpose of the Math object in JavaScript. Give an example of its usage.
3. How do you define a function in JavaScript, and what is its primary role?
4. What is the key difference between a function declaration and a function expression?
5. How do you access a specific element in an array, and what is the index of the first element?
6. Describe the primary use case for the JSON object in JavaScript.
7. What is the primary purpose of a JavaScript debugger, and how can you set a breakpoint in your code?
8. Name two common methods available for manipulating arrays in JavaScript.
9. Explain the concept of object literals in JavaScript, and provide an example of an object with multiple properties.
10. How can you access the length of a string in JavaScript, and what does it represent?

Terminal Questions

1. Explain the concept of a constructor function in JavaScript. Provide an example of a constructor function that creates objects representing books with properties such as title, author, and publication year. Additionally, demonstrate how to create multiple book objects using this constructor.
2. Write a JavaScript function that takes an array of numbers as a parameter and returns the sum of all the even numbers in the array. Provide a code example that uses this function to find the sum of even numbers in an array of your choice.

3. Describe the process of using the JavaScript debugger in a browser's DevTools to identify and fix a runtime error in your code. Provide a step-by-step explanation, including setting breakpoints, inspecting variables, and navigating through the code.
4. Explore the Date object in JavaScript. Write JavaScript code to create a Date object representing the current date and time. Then, format and display this date in the "YYYY-MM-DD HH:MM:SS" format.
5. Define an object that represents a car with properties like make, model, and year. Include a method within the object that calculates the age of the car based on the current year. Provide an example of creating a car object and calling its age calculation method.
6. Explain the concept of function scope and variable hoisting in JavaScript. Provide an example that demonstrates how variable hoisting works within a function. Additionally, describe how you can use a JavaScript debugger to inspect variable values and understand variable scope within a function.

10. ANSWERS

Self-Assessment Questions

1. A string literal in JavaScript is a direct representation of a string value enclosed in either single ('') or double ("") quotes. Example: "Hello, World!".
2. The Math object in JavaScript provides mathematical operations and constants. For example, Math.PI gives you the value of π (approximately 3.141592653589793).
3. In JavaScript, a function is defined using the function keyword, followed by a function name (if desired), parameters enclosed in parentheses, and a function body enclosed in curly braces. The primary role of a function is to group code into a reusable block that can be executed when called.
4. A function declaration uses the function keyword followed by a name, whereas a function expression can be anonymous (no name) or have a name. Function declarations are hoisted, while function expressions are not.
5. To access a specific element in an array, you use square brackets with the index of the element you want. The index of the first element in an array is 0.

6. The primary use case for the JSON object in JavaScript is to parse JSON strings into JavaScript objects using JSON.parse() and convert JavaScript objects into JSON strings using JSON.stringify().
7. A JavaScript debugger is primarily used for identifying and resolving issues (bugs) in your code. You can set a breakpoint by including the debugger; statement in your code at the desired location, and the debugger will pause execution at that point.
8. Two common methods for manipulating arrays in JavaScript are push(), which adds elements to the end of an array, and pop(), which removes and returns the last element of an array.
9. Object literals in JavaScript allow you to define objects directly in your code by specifying key-value pairs within curly braces. Example: { firstName: 'John', lastName: 'Doe' }.
10. To access the length of a string in JavaScript, you use the length property. It represents the number of characters in the string. For example, "Hello".length returns 5.

Terminal Questions

1. Refer Section 3 and 4
2. Refer Section 3 and 5
3. Refer Section 7
4. Refer Section 6
5. Refer Section 4
6. Refer Section 4 and 7